# Chapter 4
# Standard Libraries

**Abstract** In this chapter we discuss the GNU C Library and the GNU C++ Standard library. Almost all open-source applications make use of the standard libraries, and indeed it is considered good programming practice to use functions wherever possible from the standard library (unless performance or other criteria clearly dictate otherwise). The important and salient functions of the C standard library are presented, in particular the use of error return code, regular expressions, and system configuration functions are presented. In the sequel of the chapter we present some of the main features of the C++ library including Standard Template Library.

## Contents

**4.1 GNU C Library ..... 105**
**4.2 C++ Library ..... 110**
**4.3 Conclusion ..... 111**

In the coming chapters we discuss the various open source software libraries which are available for common computing tasks. These include the Boost C++ Project, Google's Perftools, ZLIB and bzip2 for data compression, HDF (Hierarchical Data Format), Berkeley db, MD5, Boehm garbage-collector, simplified Wrapper and Interface Generator (SWIG), and GNU Scheme.

## 4.1 GNU C Library

The GNU Standard C library (glibc) is the GNU implementation of the C standard library. Most C programs (except embedded platforms) use some function of the C library. C library functions are different from the system calls provided by the kernel, and execute in user-space. C library functions are provided for many of the common application processing tasks such as sorting, searching, command-line processing, FILE I/O, memory allocation, and string processing.

When using the GNU C library we should keep in mind that although it is ANSI C compliant, it also has extensions and other functions. To restrict only the ANSI

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_4, © Springer Science+Business Media, LLC 2011

subset we have to use the `-ansi` command-line argument to the compiler. The C library comprises of the header files, and the object code which implements the functions. The object codes are archived in the C library which we link against. To use a specific C library function the corresponding header file should be included in the application code. Any function name or macro defined in the ANSI standard should be treated by the application programmer as a reserved keyword, and should not be used in the application.

We discuss some of the common functions below.

1. Error conditions: C library functions return error codes in the global variable `errno` (while it is often described as a variable its implementation is defined as a modifiable lvalue). Once the application detects that the system function has encountered an error condition, only then should the error number be inspected to find out more details about the error. The various error codes are described in `errno.h`, and the application can use the `perror` (print error) function to print a message describing the error condition,
2. Memory allocation functions: memory can be allocated using the `malloc` family of functions which include besides `malloc`, `calloc`, `realloc`, and `alloca`. Memory thus allocated has to be freed using the `free` function. Alignment of memory can be assured using the `memalign` function (it takes the alignment as an additional argument alongwith size). To avoid locking a large memory region between small ones, very large memory allocations are performed using `mmmap` (memory mapping). A GNU extension called `memcheck` can check the consistency of the malloc heap (this checks for write after end, for example). Statistics about allocated memory can be obtained by using the `mallinfo` function which populates a structure of the same name. This structure contains the total number of blocks allocated. To reduce paging, a critical page of memory can be locked using the `mlock` function,
3. Character handling: these include the classic predicates of `isalpha`, `islower`, etc. One key addition is the support for *wide characters*, thus names as the characters can be wider than 1 byte in length,
4. String processing: another classic set of `strlen`, `strcmp`, etc. The memory copy functions are included in `string.h`, thus, to use `memcpy`, the application has to include `string.h`. The memory movement functions include, `memcpy`, `memmove`, and `memset`. String search functions such as `strchr` are also included. The string tokenizer function `strtok` finds use in lexical processing (see Section 13.3),
5. Searching and sorting: the GNU C library implements a binary search function and a sorting function. The search functions include *linear search*, names `lfind` and `lsearch` (`lsearch` is similar to `lfind` except that if the element is not present in the collection, it is added to it). For ordered collections a binary search function `bsearch` is implemented. Sorting is implemented using the `qsort` function which takes as input a comparison function which imposes a partial order on the elements in the universe,
6. Pattern matching: the function `fnmatch` implements pattern matching based on filenames and other text patterns. For *regular expression* or *regex* matching it uses a special data type `regex_t` defined in `regex.h` which has to be used to define

the pattern as a regular expression (see Section 13.3 for more details on regular expressions). Once a regular expression is defined it has to be *compiled* once before it can be used in multiple searches; compilation is done using the `regexcomp` function, while matching is done using the `regexec` function which returns 0 if the pattern matches,

7. FILE I/O functions: the GNU C library implements a plethora of functions dealing with IO. These include directory handling, file name query and resolution, and actual IO using streams (`FILE*`), which include functions such as `fopen`, `fclose`, `fread` and `fwrite`. The function `getline` reads an entire line from the stream. In addition to reading and writing raw bytes, the library also has functions for formatted input (`scanf`) and output (`printf`), which are well known, still an example is shown in Listing 4.1.

```
static int ReadCustomer( FILE* fp,
                         Customer* customer ) {
  static const int MAX_LEN = 1024;
  char temp[MAX_LEN];
  int rc = 0;
  rc = fscanf( fp, "Customer = [ %d %s %f %d ]\n",
               &customer->number, temp,
               &customer->amount, &customer->status );
  if( rc < 4 ) return 0;
  customer->len = strlen( temp );
  customer->name = malloc( ( customer->len+1)*sizeof( char ) );
  strcpy( customer->name, temp );
  return 1;
}
```

**Listing 4.1** Example of `scanf`

The lower level functions which operate not on streams but on actual files include `open`, `close` `read`, and `write`. The function `pread` and `pwrite` are similar to `read`, and `write`, but they accept a fourth argument which represents the file offset at which to perform the action, and these functions do not update the current file offset. The function `lseek` updates the current file offset for subsequent operations on that file descriptor. Consider the example shown in Listing 4.2.

```
  current_offset = lseek( fd, 0, SEEK_SET ); /* rewind */
  current_offset = lseek( fd, customer->number * RECORD_SIZE, SEEK_SET );
  if( current_offset == (off_t) -1 ) error_exit("lseek failed..\n");
  rc =  write( fd, &customer->number, sizeof( int ) );
  rc += write( fd, &customer->len, sizeof( int ) );
  rc += write( fd, customer->name, customer->len ) ;
  rc += write( fd, &customer->amount, sizeof( float ) );
  rc += write( fd, &customer->status, sizeof( int ) );
  current_offset = lseek( fd, ( RECORD_SIZE - rc -1 ), SEEK_CUR );
  write( fd, &customer->number, 1 ); /* marker */
  if( current_offset == (off_t) -1 ) error_exit("lseek failed..\n");
```

**Listing 4.2** Example of `lseek`

GNU C library implements fast *scatter-gather* of data which is spread in memory but contiguous on the file. Memory mapped file is implemented using `mmap` function (and has to be unmapped using the `unmap` function). A recent addition to POSIX (POSIX 1b) defines asynchronous IO using the `aio_read` and `aio_write`

functions. File control and status modes are also available using functions such as `fstat`. An example of using `fstat` is shown in Listing 4.3.

```
static void CalculateFileInformation( int fd ) {
  struct stat sb;
  int rc;
  rc = fstat( fd, &sb );
  if( ( sb.st_mode & S_IFMT ) != S_IFREG )
    error_exit("db Index should be a regular file.\n" );
  fprintf( stdout, "Preferred block size = %ld\n",
           (long) sb.st_blksize );
  fprintf( stdout, "File size = %lld\n",(long long) sb.st_size );
  fprintf( stdout, "Blocks allocated = %lld\n",
           (long long) sb.st_blocks );
  fprintf( stdout, "Last file access = %s", ctime( &sb.st_atime ) );
  fprintf( stdout, "Last file modification = %s",
```

**Listing 4.3** Example of `stat`

Directory information beyond the file level is obtained using file-system information functions defined in `unistd.h`. Files can be renamed (`rename`), or deleted (`unlink`). File size can be obtained using `stat`, and changed using `truncate` functions. Temporary files (which is guaranteed to be unique) can be opened using `tmpfile` (this function is reentrant). Interprocess communication channels can be created using `pipe`, and `popen` functions. For unrelated processes (which do not share file descriptors) a file-system file operating as a FIFO can be opened (using `mkfifo`). Thus, from shared memory, to pipes (for related processes), to FIFOs (on common file system) we come to the problem of communication channels between remote computers. This is implemented using *sockets*, which provide networking channels with similar interface as regular file descriptors.

8. Mathematical functions: including trigonometric functions are available upon including `math.h` and linking with `libm`,
9. System information :this category includes process resource usage, system information, job control, user, and groups. We have discussed resource usage functions in Section 1.1.1.1. The functions for querying the system for user, group, and other service information is done using the Name Service Switch (NSS) module. The GNU C library has functions to query group id and user id (called the persona of the process) using similarly named functions (`getuid` for get user id). An example is shown in Listing 4.4.

```
/* \file user_info.c
   \author Sandeep Koranne (C) 2010
   \description Example of using passwd structure
*/
#include <stdio.h>            // for program IO
#include <unistd.h>           // system functions
#include <string.h>           // memory allocation
#include <stdlib.h>           // library functions
#include <grp.h>              // 'group' functions
#include <pwd.h>              // 'passwd' functions
#include <sys/types.h>        // predefined types

/**
   \description print information about the process' user
*/
static void PrintSelfUserInformation(void) {
  uid_t self;
```

```
  struct passwd *self_pwd;
  struct group  *self_grp;
  char **member_of = NULL;

  self = getuid();
  printf("Self UID = %d\n", self );
  self_pwd = getpwuid( self );
  if( self_pwd == 0 ) {
    perror( "pwd retrieval failed...\n");
    exit( 1 );
  }
  printf(" Self LOGIN = %s", self_pwd->pw_name );
  printf(" Name = %s\n", self_pwd->pw_gecos );
  printf(" HOME = %s\n", self_pwd->pw_dir );
  self_grp = getgrgid( self_pwd->pw_gid );
  if( self_grp == 0 ) {
    perror( "group retrieval failed..\n");
    exit( 1 );
  }
  printf(" GROUP = %s\n", self_grp->gr_name );
  member_of = self_grp->gr_mem;
  while( *member_of ) {
    printf( "\tmember of %s\n", *( member_of ) );
  }
}

int main( int argc, char *argv[] ) {

  PrintSelfUserInformation();

  return (0);
}
```

**Listing 4.4** GNU libc example of `passwd` structure

Compiling and running this program produces the following output on my GNU/Linux system:

```
Self UID = 500
 Self LOGIN = skoranne Name = Sandeep Koranne
 HOME = /home/skoranne
 GROUP = skoranne
```

System information can be gathered using the `sysconf` function as shown in Listing 4.5.

```
/* \file page_size.c
   \author Sandeep Koranne, (C) 2010
   \description Utility to print page size
*/
#include <unistd.h>
#include <stdio.h>
int main() {
  long sz = sysconf(_SC_PAGESIZE);
  long num_phys_pages = sysconf( _SC_PHYS_PAGES );
  long num_avphys_pages = sysconf( _SC_AVPHYS_PAGES );
  printf("\n PAGE_SIZE=%ld NUM_PHYS_PAGES=%ld "
         "NUM_AV_PHYS_PAGES=%ld\n",
         sz, num_phys_pages, num_avphys_pages );
  return 0;
}
```

**Listing 4.5** GNU libc example of `sysconf`

In addition to the functions described above, the GNU C library also supports *internationalization* of programs.

## 4.2 C++ Library

The standard C++ library is called the Standard Template Library (STL). It is a library of containers, algorithms, iterators, and associated runtime support functions. STL was designed as a generic library; the data-structures of the containers are decoupled from the algorithm which operate on them. The various containers in STL are:

1. `vector`: template class of resizable array,
2. `list`: non-intrusive doubly linked list,
3. `deque`: double ended queue,
4. `set`: template class of items having partial order (implemented using height balanced trees),
5. `multisets`: same as above (sets), except allows for more than one item with the same value,
6. `map`: dictionary class with key-value semantics, where there is a partial order in the key,
7. `multimap`: same as above (map), except allows for more than one key to have same value,
8. `string`: venerable character array.

A simple example of using `std::vector` class is shown in Listing 4.6.

```
// \file stl_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Standard Template Library (STL)
#include <iostream>           // for program IO
#include <vector>             // Vector class STL
#include <cassert>            // assertion checking
#include <cstdlib>            // exit

int main( int argc, char *argv[] ) {

  std::vector<int> A(5,1); // initialize contents to 1
  std::cout << "sizeof(A) = " << sizeof(A)
            << "\n A.size() = " << A.size()
      << "\nA[0] = " << A[0];
  std::cout << std::endl;
  return (0);
}
```

**Listing 4.6** Using STL std::vector

The design of STL is based on the model of:

1. Concepts: a *concept* in C++ is a generalization of types to include semantic information,

2. Containers: abstract data types which are template based, and use the memory allocator of STL to provide non-intrusive container like data-structures, including, list, vectors, sets and maps. Containers in STL have strict requirements on the amortized runtime for insertion, deletion, and lookup. These requirements are part of the STL definition of the type, and a compliant implementation will ensure that the runtime expectations of the containers are met, e.g., `std::set` has $\log(n)$ requirement for `insert` and `find`,
3. Algorithms: algorithms in STL have two flavors. They can be predefined member functions on the containers (such as `std::map<K,V>::find`), or they can be generic functions such as `std::reverse( Range A, Range B)`. Algorithms, like containers, have strict requirement on runtime and memory, as part of their specification,
4. Iterators: iterators connect data-structures to algorithms (and vice-versa). Iterators also provide a level of indirection in the implementation of STL algorithms, which is necessary to prevent the combinatorial explosion which would ensue in its absence. For example, the `std::reverse` algorithm operates on iterator ranges. These iterators could be vector iterators, or list iterators. As long as an iterator meets the requirement (the requirement is presented as a *concept*) of the algorithm, that iterator can be passed to the algorithm.

On GNU/Linux, there are atleast two portable implementations of STL, (i) the GNU C++ library and (ii) STLport.

## 4.3 Conclusion

In this chapter we discussed the GNU C Library and the GNU C++ Standard library. The most important functions of the C standard library were presented, in particular the use of error return code, regular expressions, and system configuration functions. We also discussed the use of STL and the C++ library.